Lab 01 — Performance Profiling and Optimization using GPROF and PERF

Sorting 50 years of daily temperature data: Bubble Sort vs Quick Sort

Harsh Raj

5 August 2026

1. Scenario

A weather analytics company stores roughly 50 years of daily temperature readings (50 × 365 = **18,250 records**) and sorts them. The sort is "slower than expected". The task is to profile the existing baseline (**Bubble Sort**), compare it against the proposed replacement (**Quick Sort**), locate the bottleneck with GPROF and PERF, and make a recommendation.

Headline results

Question	Answer
Which algorithm is faster?	**Quick Sort**, by **187×** at N = 18,250 and by **12,433×** at N = 1,000,000
Bubble Sort @ 1,000,000 records	**871.8 s** (14 min 32 s) — *measured, not extrapolated*
Quick Sort @ 1,000,000 records	**0.070 s**
Top hotspot (Bubble, `-O0`)	`bubbleSort` — 73.53% of CPU time
Second hotspot (Bubble, `-O0`)	`swap` — 23.53%, over **82.9 million calls**
Top hotspot (Quick, `-O0`)	`partition` — 78.57% of CPU time
Best compiler setting	**`-O2`** — and `-O3` is **3.1× *slower*** than `-O2` here (see §8)

The single most important number in this report: bubble sort does **166,521,222 comparisons** to sort 18,250 records; quicksort does **302,516**. That ratio — **550×** — is the whole story, and no amount of compiler optimization closes it.

---

2. Files

```
Lab01/
├── mysort.c          # the program: both sorts + timing harness
├── README.md         # this document
├── report.pdf        # PDF rendering of this document
├── myreport.txt      # gprof report, -O0
├── report_O2.txt     # gprof report, -O2
├── report_O3.txt     # gprof report, -O3
└── screenshots/      # terminal captures
```

Every measured number in this report is reproduced in the tables below, so the raw intermediate files are not carried in the repository.

Reproducing the results

```
# Build at all three optimization levels
gcc -O0 -pg mysort.c -o mysort
gcc -O2 -pg mysort.c -o mysort_O2
gcc -O3 -pg mysort.c -o mysort_O3

# Run, time, and profile each (gmon.out is overwritten by every run,
# so generate each report immediately after its own run)
./mysort      && time ./mysort      && gprof ./mysort    gmon.out > myreport.txt
./mysort_O2   && time ./mysort_O2   && gprof ./mysort_O2 gmon.out > report_O2.txt
./mysort_O3   && time ./mysort_O3   && gprof ./mysort_O3 gmon.out > report_O3.txt

# Hardware counters. On WSL2 the perf wrapper cannot find a matching binary,
# so call the versioned one directly (see section 7.1).
/usr/lib/linux-tools-5.15.0-187/perf stat ./mysort

# Simulated counters - instructions, cache refs/misses, branch misses (7.2)
valgrind --tool=cachegrind --branch-sim=yes --cache-sim=yes \
         ./mysort_bench 18250 --bubble-only
```

Additional experiments cited in this report:

```
gcc -O2 mysort.c -o mysort_bench                # clean timing, no -pg overhead
gcc -O2 -DCOUNTERS mysort.c -o mysort_counters  # exact comparison/swap counts
gcc -O2 -fopenmp mysort.c -o mysort_omp         # parallel quicksort

./mysort_counters                                # section 6.3
./mysort_bench 1000000 --bubble-only             # section 9  (~15 minutes)
./mysort_bench 1000000 --quick-only --repeat 5   # section 9
./mysort_omp 4000000 --quick-only --parallel     # section 10

# Section 6.4: profile quicksort alone at a size gprof can actually sample
./mysort 4000000 --quick-only && gprof ./mysort gmon.out > report_quick_4M.txt
```

---

3. Environment

Item	Value
CPU	12th Gen Intel Core i7-1255U (6 cores / 12 threads)
Caches	L1d 48 KiB/core · L2 1.25 MiB/core (7.5 MiB total) · L3 12 MiB shared
Kernel	6.18.33.2-microsoft-standard-WSL2
GCC	11.4.0 (Ubuntu 11.4.0-1ubuntu1~22.04.3)
GNU gprof	2.38
perf	5.15.209 — software counters only; **no PMU on this kernel** (§7.1)
valgrind	3.18.1 — Cachegrind supplies the simulated counters (§7.2)

Cache sizes matter later: 18,250 `ints` = **73 KB** (fits in L2), 1,000,000 `ints` = **4 MB** (fits in L3), 4,000,000 `ints` = **16 MB** (**exceeds** the 12 MB L3). This shows up directly in the scaling curve in §9.

---

4. Program design

`mysort.c` generates the dataset, then sorts **identical copies** of it with each algorithm and verifies the output is actually in order.

Data model. Temperatures are `int` in **millidegrees Celsius** (21374 = 21.374 °C). Two reasons:

1. An `int` compare is a single instruction, so the profile measures the *sorting*, not floating-point conversion overhead.
2. 0.001 °C resolution keeps duplicate keys rare. This is not cosmetic: the Lomuto partition used here degrades toward $O(n^2)$ on runs of equal keys, so a coarse resolution (say, whole degrees) would silently change what is being measured. At 0.001 °C there are ~40,000 distinct keys.

The generator uses a triangular annual cycle (3 °C in mid-January to 27 °C in mid-July) plus pseudo-random daily noise. It deliberately avoids `<math.h>` so the exact compile line from the lab sheet — `gcc -O0 -pg mysort.c -o mysort` — links without `-lm`.

Algorithms. `swap`, `partition` and `quickSort` are kept exactly as supplied in the brief, so the profile reflects the code under study. `swap` is deliberately a real function rather than a macro: at `-O0` it stays a genuine call and appears as a hotspot; at `-O2` it is inlined and vanishes. That contrast is one of the results this lab is meant to produce (§8).

Harness note. An earlier version of the harness dispatched through function pointers (`bubble_adapter` / `quick_adapter`). That was a mistake for a profiling lab: at `-O2`/`-O3` the wrappers survive as symbols and the sort gets inlined *into* them, so gprof reported the hotspot as `bubble_adapter` — an artefact of the harness, not of the algorithm. It now dispatches through a `switch`, which inlines away cleanly.

---

5. Execution time

5.1 `time ./mysort` (N = 18,250, both sorts, best of 3)

Build	Real	User	Sys
`-O0 -pg`	**1.079 s**	1.084 s	0.000 s
`-O2 -pg`	**0.186 s**	0.183 s	0.004 s
`-O3 -pg`	**0.487 s**	0.487 s	0.004 s

`sys` time is ~0 throughout: this is a pure compute workload with one up-front allocation and no I/O in the timed region. `user` ≈ `real` confirms it is single-threaded and CPU-bound, never blocked.

5.2 In-program timing (N = 18,250)

Build	Bubble (s)	Quick (s)	Speedup	Sorted
`-O0 -pg` (profiling build)	1.0231	0.0027	381×	yes

Build	Bubble (s)	Quick (s)	Speedup	Sorted
-O2, no -pg (best of 3)	**0.174504**	**0.000932**	**187×**	yes

The -O2 row is the one to quote. The -pg row exaggerates the gap: -pg inflates bubble sort by 2.55× (§6.5) while quicksort's ~1 ms sits close to the timer's noise floor, so its 381× is an artefact of the profiling build. Throughout this report, **-O2 without -pg is the canonical configuration**; figures quoted from separate experiments may differ by a few percent, which is run-to-run variance.

5.3 A measurement trap: `clock()` under `-pg`

The first version of this harness reported quicksort's CPU time as **`0.0000 s`** while wall-clock said 0.0023 s. That is a measurement bug, not a free sort. A standalone test isolates it — the same 342 µs workload:

```
=== WITHOUT -pg ===          === WITH -pg ===
delta = 342 µs               delta = 0 µs
delta = 342 µs               delta = 0 µs
```

clock_getres() reports 1 ns for CLOCK_PROCESS_CPUTIME_ID in both builds, so this is not advertised resolution — clock() simply under-reports sub-millisecond intervals in -pg builds on this kernel. At coarser intervals (~4 ms) it agrees with getrusage and CLOCK_MONOTONIC to within 0.1%.

Consequence: this report uses CLOCK_MONOTONIC wall-clock as the primary metric, and --repeat R reports the best of R runs. Best-of is the correct summary for a benchmark — noise can only ever make a run slower, never faster, so the minimum is the closest estimate of true cost.

---

6. GPROF analysis

6.1 Bubble Sort at -O0 (`myreport.txt`) — the answer to "where is the time?"

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 73.53      0.25     0.25        1   250.00   329.84  bubbleSort
 23.53      0.33     0.08 82919103     0.00     0.00  swap
  2.94      0.34     0.01        2     5.00     5.00  is_sorted
  0.00      0.34     0.00    12273     0.00     0.00  partition
```

- **Hotspot: `bubbleSort`, 73.53% of CPU time.**
- **Second: `swap`, 23.53%, across 82,919,103 calls.** Individually swap is three instructions; collectively it is a quarter of the runtime, purely because it is *called 82.9 million times*. This is the classic profiling lesson: cost = unit cost × call count, and the call count dominates here.
- Everything else is noise. partition (all of quicksort's real work) does not register at all.

6.2 Call graph

```
                0.12    0.03       1/1           run_one
[3]     91.1    0.12    0.03       1         bubbleSort [3]
                0.03    0.00 82757197/82919103     swap [5]
-----------------------------------------------
                0.00    0.00  161906/82919103     partition [7]
```

```
                0.03    0.00 82757197/82919103     bubbleSort [3]
[5]     20.6    0.04    0.00 82919103         swap [5]
-----------------------------------------------
                                24546             quickSort [8]
[8]      0.0    0.00    0.00       1+24546   quickSort [8]
                0.00    0.00   12273/12273       partition [7]
```

Two things to read here:

1. **`swap` is shared.** The call graph splits its 82,919,103 calls by parent: **82,757,197 from `bubbleSort`** and **161,906 from `partition`** — a 511× difference from the same primitive.
2. **`1+24546` is recursion.** gprof's flat-profile `calls` column counts only *non-recursive* entries, which is why `quickSort` appears as "1" call there. The call graph shows the truth: 1 external call **+ 24,546 recursive calls**. Without reading the call graph you would wrongly conclude quicksort barely ran.

6.3 Cross-validation

The `-DCOUNTERS` build counts operations directly, independent of gprof's `mcount` instrumentation. The two agree **exactly**:

Source	Bubble swaps	Quick swaps
gprof call graph	82,757,197	161,906
`-DCOUNTERS`	82,757,197	161,906

Two independent instruments giving identical counts is good evidence neither is lying. Full operation counts at N = 18,250:

Algorithm	Comparisons	Swaps
Bubble Sort	166,521,222	82,757,197
Quick Sort	302,516	161,906
Ratio	**550×**	**511×**

Theory check. Bubble sort's expected comparisons are $\approx n^2/2 =$ **166,531,250** for n = 18,250; measured **166,521,222** — agreement to 4 significant figures. Quicksort's average case is $\approx 2n \cdot \ln n - 2.85n =$ **306,100**; measured **302,516**. Both algorithms behave exactly as the textbook analysis predicts, which is good evidence the implementations are correct.

6.4 Profiling quicksort — a subtlety worth knowing

At N = 18,250 quicksort finishes in ~1 ms, but **gprof samples every 10 ms**. Quicksort therefore collects *zero* samples and its self-time prints as `0.00` s — which does not mean it is free, only that it is below the profiler's resolution. **A profiler cannot measure what finishes faster than its sampling period.**

Re-profiling quicksort alone at N = 4,000,000 — `./mysort 4000000 --quick-only`, then `gprof` — makes it measurable:

```
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 78.57      0.33     0.33  3964001     0.00     0.00  partition
  7.14      0.36     0.03 38647333     0.00     0.00  swap
  4.76      0.40     0.02        1    20.00   380.00  quickSort
```

Quicksort's hotspot is `partition` at 78.57% — as theory predicts, since partition is where every comparison happens. Note also that gprof attributes a stray 2.38% to `bubbleSort` in this run *even though it was never called* (its `calls` column is blank) — a reminder that the time column is statistical sampling and carries error, while the call counts are exact instrumentation.

6.5 Profiling overhead

`-pg` is not free, and its cost depends entirely on how many *calls* there are:

Build	Without `-pg`	With `-pg`	Overhead
`-O0`	0.394 s	1.004 s	**2.55×**
`-O2`	0.177 s	0.173 s	~1.0× (none)

At `-O0`, `mcount` fires on all 82.7 M swap calls. At `-O2`, swap is inlined, so there is almost nothing left to instrument. **Never quote a `-pg` build's absolute timings as the program's real performance** — §5.1's numbers are for comparison between builds only; the true timings are in §9.

---

7. Hardware performance counters (PERF)

7.1 This kernel has no PMU — with evidence

`perf stat` cannot report cycles, instructions, IPC, cache references, cache misses or branch misses on this machine, and installing `perf` does not fix it. The limitation is the virtualised kernel, not the tooling: WSL2 runs under Hyper-V, which exposes **no virtual PMU** to the guest.

Direct evidence — a `perf_event_open()` probe requesting each counter:

```
HW cpu-cycles          : No such file or directory   (ENOENT)
HW instructions        : No such file or directory   (ENOENT)
HW cache-references    : No such file or directory   (ENOENT)
HW branch-misses       : No such file or directory   (ENOENT)
SW task-clock          : OK
SW page-faults         : OK
```

Every **hardware** event is rejected at the syscall; every **software** event works. `perf stat` prints `<not supported>` for exactly the rows the lab asks us to record — confirmed by running it:

```
 Performance counter stats for './mysort':

            878.03 msec task-clock:u       #    0.997 CPUs utilized
                 0      context-switches:u #    0.000 /sec
                99      page-faults:u      #  112.753 /sec
   <not supported>      cycles:u
   <not supported>      instructions:u
   <not supported>      cache-references:u
   <not supported>      cache-misses:u
   <not supported>      branch-instructions:u
   <not supported>      branch-misses:u

       0.880703603 seconds time elapsed
       0.877037000 seconds user
       0.000000000 seconds sys
```

The software counters are useful even so. Across the three builds they independently corroborate the timings in §5 and §8 — including the `-03` regression, measured here by a tool with no connection to this program's own timing code:

Build	task-clock	page-faults	context-switches
`-00 -pg`	878.03 ms	99	0
`-02 -pg`	**171.78 ms**	99	0
`-03 -pg`	497.54 ms	98	0

Zero context switches and ~99 page faults regardless of optimization level: the process is never descheduled and touches the same memory, so the entire difference between these builds is instruction cost, not system behaviour.

Getting `perf` to run at all on WSL2

Ubuntu's `perf` wrapper looks for a binary matching the *exact* running kernel and fails, suggesting packages that do not exist for WSL2 kernels:

```
WARNING: perf not found for kernel 6.18.33.2-microsoft
  You may need to install ... linux-tools-6.18.33.2-microsoft-standard-WSL2
```

Install the generic tools and call the versioned binary directly — the `perf stat` interface is stable enough across this version gap:

```
sudo apt install -y linux-tools-generic
/usr/lib/linux-tools-5.15.0-187/perf stat ./mysort
```

The probe is a dozen lines — save as `perf_probe.c`, build with `gcc -O0 perf_probe.c -o perf_probe`, and run it to reproduce the output above:

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/syscall.h>
#include <linux/perf_event.h>

static void try1(const char *name, unsigned type, unsigned long long cfg)
{
    struct perf_event_attr a;
    int fd;
    memset(&a, 0, sizeof a);
    a.size = sizeof a; a.type = type; a.config = cfg;
    a.disabled = 1; a.exclude_kernel = 1; a.exclude_hv = 1;
    fd = syscall(__NR_perf_event_open, &a, 0, -1, -1, 0);
    printf("%-22s : %s\n", name, fd < 0 ? strerror(errno) : "OK");
    if (fd >= 0) close(fd);
}

int main(void)
{
    try1("HW cpu-cycles",    PERF_TYPE_HARDWARE, PERF_COUNT_HW_CPU_CYCLES);
    try1("HW instructions",  PERF_TYPE_HARDWARE, PERF_COUNT_HW_INSTRUCTIONS);
    try1("HW cache-refs",    PERF_TYPE_HARDWARE, PERF_COUNT_HW_CACHE_REFERENCES);
    try1("HW branch-misses", PERF_TYPE_HARDWARE, PERF_COUNT_HW_BRANCH_MISSES);
```

```
    try1("SW task-clock",    PERF_TYPE_SOFTWARE, PERF_COUNT_SW_TASK_CLOCK);
    try1("SW page-faults",   PERF_TYPE_SOFTWARE, PERF_COUNT_SW_PAGE_FAULTS);
    return 0;
}
```

To collect real hardware counters, run on native (non-virtualised) Linux:

```
perf stat -e task-clock,context-switches,page-faults,cycles,instructions,\
cache-references,cache-misses,branch-instructions,branch-misses ./mysort
```

7.2 Filling the gap with Cachegrind

Cachegrind recovers everything the missing PMU costs us except cycles, because it *simulates* a cache and branch-predictor model instead of reading hardware:

```
sudo apt install -y valgrind
valgrind --tool=cachegrind --branch-sim=yes --cache-sim=yes \
         ./mysort_bench 18250 --bubble-only
valgrind --tool=cachegrind --branch-sim=yes --cache-sim=yes \
         ./mysort_bench 18250 --quick-only
```

Read these as architecturally faithful but not cycle-exact — a model of this cache hierarchy, not a measurement of it. Cachegrind also runs ~50× slower than native, which is why N stays at 18,250.

Cycles and IPC remain genuinely unobtainable. Both need a real cycle counter, and no simulator can supply one. Those two rows stay empty rather than being filled with a guess.

7.3 Performance comparison table

The comparison the lab asks for, at N = 18,250. Timing is `-O2` without `-pg`; counters are from Cachegrind; the hotspot is from gprof.

Metric	Bubble Sort	Quick Sort	Ratio
Execution time	0.174504 s	0.000932 s	**187×**
Instructions (`I refs`)	2,085,040,959	8,050,471	**259×**
Cache references (`D refs`)	500,186,897	2,528,599	**198×**
Cache misses (`D1`)	5,317,138	10,479	**507×**
Last-level misses	5,874	5,894	**1.00×**
Branches	333,822,603	1,384,092	**241×**
Branch misses	18,233,440	116,034	**157×**
Branch miss *rate*	5.5%	8.4%	0.65×
Comparisons	166,521,222	302,516	**550×**
Swaps	82,757,197	161,906	**511×**
CPU cycles	*no PMU (§7.1)*	*no PMU*	—
IPC	*no PMU (§7.1)*	*no PMU*	—
Hotspot function	`bubbleSort` 73.53%(swap 23.53%)	`partition` 78.57%	—

Three things in this table are worth more than the headline ratio.

Last-level misses are identical — 5,874 vs 5,894. At N = 18,250 the array is 73 KB, which fits comfortably in the 1.25 MB L2. Both algorithms pay the same compulsory misses to load it once and neither thrashes. **Cache behaviour is not why bubble sort loses at this size** — it loses purely on instruction count. That is worth stating plainly, because "it must be a cache problem" is the reflexive explanation for slow code, and here it is measurably wrong. (Cache *does* become decisive at large N — see §9, where quicksort drifts above its n log n prediction once the working set passes the 12 MB L3.)

Bubble sort has the *better* branch prediction rate — 5.5% vs 8.4% — and still loses by 157× on absolute mispredictions. Quicksort's partition comparison is close to a coin flip and genuinely hard to predict, while bubble sort's `arr[j] > arr[j+1]` becomes increasingly predictable as the data gets sorted. Bubble sort is *better* at the thing branch predictors reward, and it does not help, because it executes 241× more branches. A good rate on a huge volume loses to a poor rate on a small one.

2.085 billion instructions versus 8.05 million. This is the instruction-count row the PMU could not give us, and it lands where the comparison counts said it would.

---

8. Compiler optimizations

Best of 9 runs, no `-pg`, wall-clock seconds:

Level	Bubble (N=18,250)	vs `-O0`	Quick (N=1,000,000)	vs `-O0`
`-O0`	0.337 s	1.00×	0.113 s	1.00×
`-O1`	0.165 s	**2.04×**	0.0548 s	**2.06×**
`-O2`	0.166 s	**2.03×**	0.0587 s	1.93×
`-O3`	0.520 s	**0.65× (slower!)**	0.0623 s	1.82×

8.1 What `-O2` actually did

The gprof reports show the mechanism directly. At `-O0` the profile has four entries; at `-O2`:

```
100.00      0.18     0.18        1   180.00   180.00  bubbleSort
  0.00      0.18     0.00    12273     0.00     0.00  partition
```

`swap` has disappeared entirely — inlined into its callers. 82.9 million function calls, each with prologue, epilogue, stack frame and pointer indirection, became inline register operations. That single transformation is most of the 2× gain, and it is also why `-pg` overhead collapses to zero (§6.5).

At `-O3` the profile degrades further for profiling purposes: `bubbleSort` itself is inlined into `run_one`, so gprof attributes **100% to `run_one`** and the algorithm's own symbol is gone. **Aggressive inlining and profiling are in tension** — the more the compiler optimizes, the less the profile resembles the source. This is why production profiling usually uses `-O2 -g -fno-omit-frame-pointer` rather than `-O3`.

8.2 `-O3` is 3.1× *slower* than `-O2` — root cause

This is the most interesting result in the lab, and it is a real effect, not measurement noise or a `-pg` artefact. Isolating it:

Build	Bubble, N=18,250 (best of 5)
`-O2`	0.171 s
`-O3`	0.588 s
`-O3 -fno-tree-vectorize`	**0.205 s**
`-O3 -fno-if-conversion`	0.479 s

Disabling the vectorizer recovers nearly all the loss, which points straight at auto-vectorization. Confirmed in the generated assembly for `bubbleSort`:

	-O2	-O3
SIMD register refs (`xmm`/`ymm`)	**0**	**6**
Shuffle instructions (`pshufd`)	0	2

`-O3` enables `-ftree-slp-vectorize`, which tries to pack bubble sort's inner loop into SIMD registers. But bubble sort's inner loop carries a **loop-carried dependency** — `arr[j+1]` written on iteration *j* is read on iteration *j+1* — so the vectorizer can only proceed by adding shuffles and extra data movement around it. The result is correct (output verified sorted) but the shuffle overhead costs more than the vector width saves.

Lesson: `-O3` is not "`-O2` but better". It enables transformations that are profitable on average and harmful on specific code shapes. Always measure.

---

9. Effect of problem size

Wall-clock seconds, `-O2`, no `-pg`, best-of measurements:

N	Bubble Sort	Ratio	Quick Sort	Ratio
1,000	0.000424	—	0.000017	—
2,000	0.001894	4.47×	0.000061	3.59×
4,000	0.007807	4.12×	0.000168	2.75×
8,000	0.031517	4.04×	0.000356	2.12×
18,250	0.174504	5.54×	0.000932	2.62×
32,000	0.606400	3.47×	0.001701	1.83×
64,000	2.950611	4.87×	0.003499	2.06×
128,000	13.437407	4.55×	0.007202	2.06×
256,000	57.313804	4.27×	0.015157	2.10×
500,000	—	—	0.029352	1.94×
1,000,000	**871.760620**	15.21×	0.070114	2.39×
2,000,000	—	—	0.154128	2.20×
4,000,000	—	—	0.352225	2.29×
8,000,000	—	—	0.879219	2.50×

(Bubble sort ratios spanning a non-doubling step — 8,000 → 18,250 and 256,000 → 1,000,000 — are larger because *n* more than doubles there.)

Bubble Sort — quadratic, confirmed. Every doubling of *n* multiplies time by ≈ 4 (measured 4.04–4.87 across the clean doubling steps). The strongest single check is the last step: 256,000 → 1,000,000 is 3.906× more data, and pure n^2 predicts a **15.26×** slowdown. Measured: **15.21×**. **871.8 seconds** to sort one million records.

Quick Sort — n log n, confirmed. Every doubling multiplies time by ≈ 2.1, against a theoretical 2.11× in this range. The two smallest sizes (1,000 and 2,000) show inflated ratios only because 17–61 µs is near the timer's noise floor.

Beyond ~2 M elements the ratio drifts above prediction (2.29×, 2.50× versus ~2.09× predicted). At 8 M elements the array is **32 MB**, well past the 12 MB L3 cache, so memory traffic — not comparison count — starts to set performance. This is exactly the regime where cache behaviour matters more than asymptotics.

The gap widens without limit. The ratio is not a constant factor; it grows as O(n / log n):

N	Bubble	Quick	Bubble / Quick
1,000	0.000424 s	0.000017 s	25×
18,250	0.174504 s	0.000932 s	187×
256,000	57.31 s	0.015157 s	3,781×
1,000,000	871.76 s	0.070114 s	**12,433×**

---

10. Feasibility of parallel execution

Implemented as OpenMP tasks (`quickSortOMP` in `mysort.c`, build with `-fopenmp`). The two recursive calls touch disjoint halves of the array, so they are independent and safe to run concurrently. Two guards prevent collapse under task-creation overhead: stop spawning below a depth of ~$\log_2$(threads), and sort subarrays under 50,000 elements serially.

Measured at N = 4,000,000 on 12 threads:

Version	Wall (s)	CPU (s)	Speedup
Serial quicksort	0.5537	0.5560	1.0×
OpenMP quicksort	**0.2267**	1.7019	**2.4×**

Verdict: worthwhile but sharply limited. 2.4× on 12 threads is 20% efficiency. The reason is structural, not an implementation defect:

- **The first partition is inherently serial.** It touches all *n* elements before any parallelism exists. That alone is ~50% of the total comparison work at the top two levels, and by Amdahl's law a 50% serial fraction caps speedup at 2× regardless of core count. The measured 2.4× is consistent with this.
- **Memory bandwidth saturates.** Sorting is memory-bound at 16 MB; extra threads compete for the same L3 and DRAM bandwidth.
- **CPU time rose 3.1×** (0.556 → 1.702 s) — parallelism bought latency at a real cost in total work, which matters on shared or battery-powered machines.
- This CPU is 2 performance cores + 8 efficient cores, so 12 threads are not 12 equal cores.

For genuinely large datasets, a parallel **merge** sort or sample sort scales better, because it parallelises the partitioning step itself rather than only the recursion. **Bubble sort is not usefully parallelisable** in this form — each pass depends on the previous one. Its parallel relative, odd–even transposition sort, still needs O(n) parallel steps and remains far worse than serial quicksort.

---

11. Answers to the lab questions

1. Which sorting algorithm performed better? Explain. Quick Sort, overwhelmingly — **187×** at N = 18,250 and **12,433×** at N = 1,000,000. The cause is algorithmic complexity, not implementation quality: bubble sort is O(n^2) and performed 166,521,222 comparisons on the 18,250-record dataset, while quicksort is O(n log n) and performed 302,516 — a **550×** difference in work. Bubble sort compares only adjacent elements, so each swap moves an element one position; quicksort's partitioning moves elements a long way toward their final position, so it never revisits the same pair.

2. Which function consumed the maximum execution time? For the baseline, `bubbleSort` at **73.53%** of CPU time, with swap second at **23.53%** across **82,919,103 calls**. For quicksort (profiled at N = 4,000,000 so gprof can resolve it), `partition` at **78.57%** — expected, since partition

is where every comparison happens. At `-O2` `swap` disappears from the profile entirely because it is inlined; at `-O3` even `bubbleSort` is inlined into its caller.

3. How does execution time change with increasing input size? Bubble sort quadruples per doubling of n (measured 4.04–4.87×, theory 4×); quicksort roughly doubles (measured ~2.1×, theory 2.11×). The sharpest check: going from 256,000 to 1,000,000 records, n^2 predicts a 15.26× slowdown and the measurement gives 15.21×. The gap therefore widens without bound — 25× at N = 1,000 but **12,433×** at N = 1,000,000. Beyond ~2 M elements quicksort drifts slightly above its n log n prediction as the working set (32 MB at N = 8 M) exceeds the 12 MB L3 cache and memory bandwidth becomes the limit.

4. How did compiler optimizations (-O2 and -O3) improve performance? `-O2` gave a **2.03×** speedup on bubble sort, almost entirely by inlining `swap` — eliminating 82.9 M function calls, which is visible as `swap` disappearing from the `-O2` gprof report. `-O3` **made it 3.1× worse** (0.166 s → 0.520 s): it enables SLP auto-vectorization, which emits `xmm` registers and `pshufd` shuffles into a loop that has a loop-carried dependency. `-fno-tree-vectorize` recovers the loss, confirming the cause. Compiler optimization is worth ~2×; the algorithm change is worth ~10,000×.

5. Which algorithm would you recommend for large datasets? Justify. Quick Sort, with the reservations below. At 1,000,000 records it finishes in 0.070 s against bubble sort's 871.8 s — the difference between interactive and unusable. Bubble sort should be retained only as a teaching baseline.

Caveats worth stating rather than hiding:

- The supplied Lomuto partition takes the **last element as pivot**, which degrades to **$O(n^2)$ time and O(n) recursion depth on already-sorted or reverse-sorted input** — and weather data very often arrives sorted by date. A production version needs median-of-three or a randomized pivot, plus recursion on the smaller side first to bound stack depth.
- Quicksort is **not stable**. If records carry more than a temperature (station ID, timestamp), equal-key ordering is not preserved.
- For the actual business problem — **18,250 records, sorted in under 1 ms** — the honest recommendation is `qsort()` from libc, or `std::sort`. It is already written, already tested, and uses introsort, which falls back to heap sort when quicksort's recursion goes bad. Writing a custom quicksort is only justified if profiling proves the library call is the bottleneck, and at 0.9 ms it is not.

---

12. Recommendations

1. **Replace bubble sort with quicksort** — 187× today, and the margin grows with the dataset.
2. **Ship with `-O2`, not `-O3`.** `-O3` is measurably slower on this code. Treat `-O3` as a hypothesis to be benchmarked, never a default.
3. **Fix the pivot before production.** Last-element pivot on date-ordered weather data is a latent $O(n^2)$ and a stack-overflow risk.
4. **Prefer `qsort()`** unless profiling proves otherwise.
5. **Parallelise only above ~1 M records.** Below that, serial quicksort's sub-millisecond runtime makes 2.4× irrelevant.
6. **Profile at `-O2`, not `-O0` or `-O3`.** `-O0` inflates call overhead by 2.55×; `-O3` inlines the functions you are trying to observe.